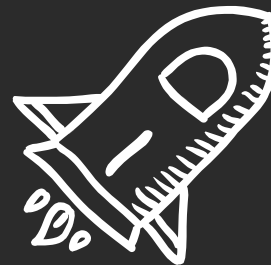




Semana 12

¡Bienvenidos!



Temas de hoy

1

Vistas y URLs en
Django

2

Definición de vistas.

3

Manejo de URLs y
rutas.

1

Vistas y URLs en Django

Imaginate que tenés una app web, y cada parte de tu sitio es como una ciudad distinta.

Ahora, ¿cómo haces para que los usuarios lleguen a la ciudad correcta sin perderse por el camino?

Ahí es donde entran las vistas y URLs. En Django, son como... el GPS de tu aplicación.

¡Vamos a ver más sobre esto!

Vistas en Django

¿Qué es una vista?

En Django, una vista es una función o clase que recibe una solicitud (request) y devuelve una respuesta (response). Las vistas contienen la lógica para mostrar lo que los usuarios ven en el navegador.

Django organiza las vistas dentro del archivo views.py de la aplicación general y también (opcionalmente) de cada aplicación. Las vistas procesan las solicitudes web y generan las respuestas adecuadas (puede ser HTML, JSON, o una redirección, entre otros).

→ Tipos de Vistas:

Vistas basadas en funciones (Function-Based Views - FBV):

Son simplemente funciones de Python que reciben un objeto request y devuelven una respuesta.

Vistas basadas en clases (Class-Based Views - CBV):

Son clases que permiten reutilizar código de forma más estructurada y modular.

URLS en Django

¿Qué es un URLconf?

Las URLs en Django son las rutas que se definen para que la aplicación web sepa cómo responder a las solicitudes HTTP que hacen los usuarios en el navegador. Es decir, cuando un usuario ingresa una URL en su navegador, Django utiliza el sistema de URLs para determinar qué vista debe procesar esa solicitud.

- En términos simples, las URLs mapean las solicitudes entrantes (como `https://sitio.com/home/`) con las funciones o clases (vistas) que devuelven una respuesta adecuada.
- En proyectos grandes, cada aplicación tiene su propio archivo `urls.py`, y las rutas de cada aplicación se incluyen en el archivo `urls.py` del proyecto principal.

A esto lo veremos en las siguientes diapositivas más explicado, no te preocupes.

1

Definición de vistas

Ahora imaginate que tu aplicación web es una obra de teatro.

Tenés el escenario (tu HTML), el público (los usuarios) y los actores (los datos que mostrás).

Pero falta lo más importante: el director que le dice a cada actor qué hacer y cuándo entrar!

En Django, ese director son las **vistas**.

Definición de vistas

Vistas basadas en funciones (FBV)

Una vista basada en función es una función Python que toma al menos un parámetro request. Generalmente, devuelve una respuesta HttpResponse o una plantilla HTML renderizada.

```
1 from django.http import HttpResponse
2
3 def home_view(request):
4     return HttpResponse("Bienvenido a la página principal")
```

En este ejemplo:

home_view es la función que representa la vista.

El parámetro **request** es obligatorio y representa la solicitud que llega al servidor.

HttpResponse genera una respuesta simple de texto.

O podemos hacerlo de la siguiente manera:

```
1 from django.shortcuts import render
2
3 def about_view(request):
4     return render(request, 'about.html')
```

Acá usamos render para devolver una plantilla HTML, en este caso 'about.html'.

Definición de vistas

Vistas basadas en funciones (FBV)

Vistas basadas en funciones (FBV)

Si nos vamos a la definición de render:

```
def render(  
    request, template_name, context=None, content_type=None, status=None, using=None  
):  
    """  
    Return an HttpResponse whose content is filled with the result of calling  
    django.template.loader.render_to_string() with the passed arguments.  
    """  
    content = loader.render_to_string(template_name, context, request, using=using)  
    return HttpResponse(content, content_type, status)
```

Vemos que hay 2 parámetros que son necesarios pasarlos: `request` y `template_name`, pero también tenemos los opcionales, de los cuales hay uno llamado `context` que se usa para pasar datos al template y poder mostrarlos, por ejemplo:

```
def mostrar_objetos(request):  
    modulos = Modulo1.objects.all()  
    context = {  
        'modulos' : modulos  
    }  
    template_name = 'modulo1/listar_objetos.html'  
    return render(request = request, template_name = template_name, context = context)
```

O para un solo objeto:

```
def mostrar_objeto(request, id):  
    modulo = Modulo1.objects.get( id = id)  
    context = {  
        'modulo' : modulo  
    }  
    template_name = 'modulo1/detalle_objeto.html'  
    return render(request = request, template_name = template_name, context = context)
```

Definición de vistas

Vistas basadas en clases (CBV)

Las vistas basadas en clases son más flexibles y reutilizables. Son útiles cuando una vista necesita manejar diferentes tipos de solicitudes HTTP como GET, POST, etc.

```
1 from django.http import HttpResponse
2 from django.views import View
3
4 class HomeView(View):
5     def get(self, request):
6         return HttpResponse("Bienvenido a la página principal")
```

En este ejemplo usamos la clase View de Django y definimos un método get para manejar las solicitudes GET.

O podemos hacerlo de la siguiente manera:

```
1 from django.views.generic import TemplateView
2
3 class AboutView(TemplateView):
4     template_name = 'about.html'
```

TemplateView es una vista genérica que solo necesita una plantilla.

Definición de vistas

Vistas basadas en clases (CBV)

```
from django.urls import reverse_lazy
from django.views.generic.edit import CreateView, UpdateView, DeleteView
from django.views.generic import ListView, DetailView

from apps.modulo1.models import Modulo1

class CrearModulo1(CreateView):
    model = Modulo1
    fields = ['nombre', 'descripcion']
    template_name = 'modulo1/agregar_objeto.html'
    success_url = reverse_lazy('inicio')

class ActualizarModulo1(UpdateView):
    model = Modulo1
    fields = ['nombre', 'descripcion']
    template_name = 'modulo1/actualizar_objeto.html'
    success_url = reverse_lazy('inicio')

class EliminarModulo1(DeleteView):
    model = Modulo1
    template_name = 'modulo1/confirmar_eliminar_objeto.html'
    success_url = reverse_lazy('inicio')
```

```
class ListarModulo1(ListView):
    model = Modulo1
    template_name = 'modulo1/listar_objetos.html'
    context_object_name = 'modulos'
    paginate_by = 3

    def get_context_data(self) :
        context = super().get_context_data()
        modulos = Modulo1.objects.all()
        context['modulos'] = modulos
        return context

    def get_queryset(self):
        query = self.request.GET.get('buscador')
        queryset = super().get_queryset()

        if query:
            queryset = queryset.filter(nombre__icontains = query)
        return queryset.order_by('nombre')
```

3

Manejo de URLs y rutas

Ahora bien, Django es como el Google Maps de tu aplicación: cada vez que alguien ingresa una URL, tiene que saber exactamente a dónde mandarlo.

Pero... cómo lo hace?

Fácil: vos le decís qué vista (recordemos que son los que muestran el contenido) tiene que cargar para cada dirección.

Manejo de URLs y rutas

Conceptos

En Django, el manejo de URLs es fundamental para definir cómo los usuarios interactúan con tu aplicación web. Las URLs son las direcciones que un usuario ingresa en el navegador para acceder a diferentes secciones de tu sitio, y el archivo **urls.py** es donde configuramos esas rutas para que el sistema sepa qué hacer con cada una.

Cada proyecto tiene al menos un archivo **urls.py**, que es el encargado de vincular las URLs con las vistas que deben ejecutarse. Este archivo es esencialmente un mapa de rutas, y por cada ruta especificada, Django sabe a qué vista llamar para entregar la respuesta adecuada al usuario.

Cada ruta tiene tres partes clave:

Patrón de URL: Es la dirección que el usuario va a escribir en el navegador, como 'productos/' o 'contacto/'.

Vista: Es la función o clase que manejará la solicitud cuando un usuario acceda a esa URL. Por ejemplo, si el usuario accede a /productos/, Django llamará a la vista `views.productos`.

Nombre (opcional): Es un alias que le damos a la URL para referirnos a ella fácilmente en otras partes del código, como las plantillas.

Manejo de URLs y rutas

Ejemplo

proyecto/urls.py:

```
from django.contrib import admin
from django.urls import include, path

from .views import index

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", index, name='index' ),
    path("modulo1/", include('apps.modulo1.urls'))
]
```

proyecto/apps/modulo1/urls.py

```
from django.urls import path

from .views import CrearModulo1, ListarModulo1, ActualizarModulo1, EliminarModulo1

app_name = 'apps.modulo1'

urlpatterns = [
    path("crear_modulo/", CrearModulo1.as_view(), name='crear_modulo'),
    path("actualizar_modulo/<int:pk>", ActualizarModulo1.as_view(), name='actualizar_modulo'),
    path("listar_modulos/", ListarModulo1.as_view(), name='listar_modulos'),
    path("eliminar_modulo/<int:pk>", EliminarModulo1.as_view(), name='eliminar_modulo'),
]
```

En estos ejemplos definimos como se mapean las urls (con vistas basadas en clases) de nuestro proyecto, con 'include()' podemos hacer que las urls definidas en el archivo, con la ubicación que se especifica dentro, sean parte de nuestro proyecto pero modularizadas para cada aplicación.

Manejo de URLs y rutas

Buenas prácticas



- **Agrupá y nombrá las URLs:** Usá el argumento name en las rutas para hacer referencia a ellas desde las plantillas y el código Python sin necesidad de hardcodear las URLs. Así, si cambiás una ruta, solo necesitás modificar el urls.py, no todo tu proyecto.
- **Evitá duplicar rutas:** Si varias URLs comparten una misma vista o lógica, tratá de usar parámetros dinámicos para evitar tener rutas redundantes.
- **Separá por aplicaciones:** Si tu proyecto tiene varias funcionalidades, es recomendable crear diferentes archivos urls.py en cada aplicación para mantener el código más ordenado.



Lo que vimos hoy 🤔

Vimos cómo las URLs en Django son como las direcciones que los usuarios escriben en el navegador para llegar a distintas partes de la app. Para cada URL, definimos una vista, que es como el recepcionista que recibe la solicitud y decide qué mostrar en pantalla. Las vistas pueden ser funciones o clases, según lo que necesitemos. También aprendimos a manejar las rutas en el archivo `urls.py`, que es el mapa que conecta las URLs con sus vistas correspondientes.



**¡Nos vemos
En la próxima
clase!**

